DATA SCIENCE / GUIA TÉCNICO EP

# Orientação a Objetos

Classes, invariantes, composição, herança e dataclasses

Objetivo. Estudo aprofundado para apoiar os notebooks de Programação Python, com explicações, exemplos executáveis, exercícios e cuidados para ambientes reais.

Mapa de estudo. 1. Objetos, estado e invariantes • 2. Dataclasses e validação • 3. Composição e herança • 4. Propriedades e testes de comportamento • 5. Métodos de classe e métodos estáticos • 6. Representação e igualdade de objetos • 7. Design orientado a contratos • 8. Projeto integrador e testes

## Como estudar

Leia o princípio, execute o exemplo em um ambiente preparado, modifique entradas e resolva o desafio antes de avançar. Os exemplos são didáticos e não alteram os notebooks originais.

## Relação com o acervo

Este guia é complementar à trilha de seis notebooks. A documentação oficial das bibliotecas é a fonte de referência para APIs e mudanças de versões.

## 01 Objetos, estado e invariantes

Classes conectam estado e comportamento; __init__ prepara atributos e métodos aplicam regras. Invariantes são condições que devem permanecer verdadeiras durante o ciclo de vida do objeto.

Exemplo executável

```
class Conta:
    def __init__(self, saldo=0):
        if saldo < 0:
            raise ValueError("Saldo invalido")
        self.saldo = saldo
    def depositar(self, valor):
        if valor <= 0:
            raise ValueError("Valor invalido")
        self.saldo += valor
```

Atenção técnica. Evite expor estado que possa ser alterado sem respeitar validações de domínio.

Exercício de consolidação. Teste depósito negativo e saldo inicial inválido.

## 02 Dataclasses e validação

@dataclass reduz boilerplate em modelos de dados simples; __post_init__ valida após a criação dos atributos. Tipos anotados não são verificados automaticamente em tempo de execução.

Exemplo executável

```
from dataclasses import dataclass
@dataclass
class Produto:
    nome: str
    preco: float
    def __post_init__(self):
        if self.preco < 0:
            raise ValueError("Preco invalido")

assert Produto("Livro", 45).preco == 45
```

Atenção técnica. Dataclass não substitui desenho de domínio; classes com comportamento relevante devem explicitar contratos.

Exercício de consolidação. Inclua quantidade e verifique se o valor não é negativo.

## 03 Composição e herança

Composição modela relações tem-um; herança modela é-um e exige substituibilidade sem quebra de contrato. Prefira composição quando classes só compartilham parte da implementação.

Exemplo executável

```
class Carrinho:
    def __init__(self):
        self.itens = []
    def adicionar(self, produto):
        self.itens.append(produto)
    @property
    def total(self):
        return sum(p.preco for p in self.itens)
```

Atenção técnica. Evite herança para reutilização incidental; composição geralmente reduz acoplamento.

Exercício de consolidação. Explique por que Carrinho não deve herdar de Produto.

## 04 Propriedades e testes de comportamento

@property oferece acesso semelhante a atributo com cálculo controlado. Testes verificam contratos, não detalhes internos. Separe modelo, persistência e apresentação para facilitar manutenção.

Exemplo executável

```
c = Carrinho()
c.adicionar(Produto("Livro", 45))
c.adicionar(Produto("Caneta", 2.5))
assert c.total == 47.5
assert Carrinho().total == 0
```

Atenção técnica. Acessar c.total é diferente de c.total(); propriedades não são métodos invocados com parênteses.

Exercício de consolidação. Adicione remoção de produtos e teste consistência do total.

## 05 Métodos de classe e métodos estáticos

classmethod recebe cls e pode construir instâncias alternativas. staticmethod agrupa função utilitária relacionada ao tipo, mas sem acesso automático a self ou cls. Escolha pela necessidade real de estado de classe.

Exemplo executável

```
class Coordenada:
    def __init__(self, x, y):
        self.x, self.y = x, y
    @classmethod
    def origem(cls):
        return cls(0, 0)
assert Coordenada.origem().x == 0
```

Atenção técnica. Evite usar staticmethod apenas como organização de funções que não pertencem à classe.

Exercício de consolidação. Implemente um construtor alternativo a partir de uma string x,y.

## 06 Representação e igualdade de objetos

__repr__ oferece representação para inspeção; __eq__ define igualdade de valores. dataclasses geram métodos úteis por padrão, mas decisões de mutabilidade e identidade devem ser explícitas. Objetos mutáveis são geralmente inadequados como chaves de dict.

Exemplo executável

```
from dataclasses import dataclass
@dataclass(frozen=True)
class Ponto:
    x: int
    y: int
p = Ponto(1,2)
assert p == Ponto(1,2)
```

Atenção técnica. frozen=True impede atribuição simples, mas não torna automaticamente imutáveis objetos mutáveis contidos nos campos.

Exercício de consolidação. Compare identidade com igualdade em duas instâncias equivalentes.

## 07 Design orientado a contratos

Classes coesas têm poucas responsabilidades relacionadas. Composição permite trocar dependências e testar colaborador sem banco ou rede reais. A herança deve obedecer à substituição: um subtipo não pode quebrar garantias públicas do tipo base.

Exemplo executável

```
class FreteFixo:
    def calcular(self, itens):
        return 10
class Pedido:
    def __init__(self, frete):
        self.frete = frete
    def custo_frete(self):
        return self.frete.calcular([])
assert Pedido(FreteFixo()).custo_frete() == 10
```

Atenção técnica. Dependências criadas internamente tornam testes mais difíceis e podem acoplar lógica a infraestrutura.

Exercício de consolidação. Crie FreteGratis e injete em Pedido sem modificar Pedido.

## 08 Projeto integrador e testes

O carrinho de produtos exige decisões de invariantes: preço não negativo, quantidades válidas e somatório consistente. Faça testes após cada operação, incluindo carrinho vazio, repetição, exclusão e erro. Separe modelo dos arquivos de persistência.

Exemplo executável

```
c = Carrinho()
assert c.total == 0
c.adicionar(Produto("A", 2))
c.adicionar(Produto("B", 3))
assert c.total == 5
```

Atenção técnica. Não apresente um teste de integração como teste unitário isolado quando envolve vários componentes e arquivos.

Exercício de consolidação. Acrescente remover_item e verifique total após exclusão.

## Checklist de revisão

| Verifique | Evidência |
|---|---|
| Conceito | Consegue explicar sem copiar o código? |
| Execução | O exemplo produz o resultado esperado? |
| Casos-limite | Entradas inválidas e extremas foram testadas? |
| Reprodutibilidade | O kernel e as dependências estão registrados? |

## Referências técnicas

Python: docs.python.org/3/ • NumPy: numpy.org/doc/stable/ • Matplotlib: matplotlib.org/stable/ • Pandas: pandas.pydata.org/docs/

Material didático autoral EP. A utilização de código e materiais de terceiros está sujeita às respectivas licenças.